

LumiWeave 系统评估与优化指南

对话交互与画布界面 (Canvas) 融合架构设计

1. 当前项目架构评估

从当前的 LumiWeave 系统结构来看，项目已经具备了完整的前后端分离架构，包含 Agent、AI 模型调度、Prompt 学习引擎、渲染器接口 (Renderer) 以及技能插件 (Skills) 等核心模块。当前将界面精简为**纯对话性 (Conversational)** 模式，具有以下优势与局限：

- 优势：** 降低了用户的上手门槛。对话式 UI (ChatPanel) 线性且直观，非常适合单步问答、简单的工具调用以及信息检索任务。
- 局限：** 对于多智能体协作、复杂的提示词链 (Prompt Chaining)、带有条件分支的逻辑控制，纯对话界面缺乏“空间感”。用户难以直观地审查和修改中间步骤的数据流向，也无法高效地进行工作流复用。

2. 交互体验优化指导：Chat + Canvas 融合模式

为了兼顾易用性与专业深度，建议采用 **对话 + 画布双轨制 (Dual-Track UI)**。这并非要放弃当前的对话界面，而是让两者形成互补。

💡 设计理念借鉴

在设计画布连线与节点流转时，可以参考类似 ComfyUI 的节点逻辑体系。通过将底层的 LLM 调用、Prompt 处理、API 请求封装成独立的黑盒节点，通过连线决定数据流，实现工作流的直观编排与复用。

2.1 界面布局建议

- **左侧抽屉 / 浮窗视图：**保留现有的 ChatPanel，作为自然语言指令的入口。用户可以通过对话唤起特定任务。
- **右侧主视图：**引入无限画布 (Infinite Canvas)。当用户在对话中触发复杂任务时，系统在后台自动生成对应的节点工作流，并映射到右侧画布上进行可视化。
- **“弹出至画布 (Eject to Canvas)”：**在聊天记录的消息气泡旁增加操作按钮，允许用户将某次对话的背后逻辑“展开”至画布，进行精细化参数微调。

3. 画布界面 (Canvas) 补充架构设计

在前端补充画布界面，需要定义清晰的节点系统与数据结构，确保与后端的 `agent/router.py` 和 `skills/manager.py` 完美对接。

3.1 核心节点类型设计 (Node Schema)

节点分类	对应 LumiWeave 模块	功能描述
Input/ Output	Frontend / ChatPanel	处理用户的初始输入变量，以及将最终结果格式化输出（文字、媒体流等）。
LLM Engine	backend/app/ai/ client.py	大模型推理节点。可配置模型参数 (Temperature, Top-P) 以及使用的模型版本。
Prompt Template	prompt_learning/	提示词构建节点。支持变量注入，可串联上下文记忆或动态提取的知识库片段。
Skill / Tool	backend/app/skills/	外部工具调用节点（如检索、代码执行、渲染任务触发）。
Router / Logic	backend/app/agent/ router.py	条件判断与路由分发节点，用于实现多 Agent 协同和复杂流程的控制。

3.2 数据流向与状态管理

画布上的每一个节点都应被视为一个有向无环图 (DAG) 中的独立函数。状态管理是画布界面的核心：

- **前端状态：** 推荐引入 React Flow 库来快速构建节点与连线交互。使用 Zustand 管理全局的节点位置、连线状态及节点内的参数配置。
- **后端解析：** 前端画布导出 JSON 格式的 DAG 数据提交至后端。后端需新增一个 Workflow Engine 模块，按拓扑排序依次执行节点任务，并将中间执行状态通过 WebSocket 实时推送到前端，点亮对应的节点。

4. 落地与实施步骤

Phase 1: 基础设施搭建

在前端 `frontend/src/components/` 目录下新增 `WorkflowCanvas.tsx`。集成 React Flow 依赖，实现基础的节点拖拽、连线和删除功能。定义基本的 JSON Schema 用于描述工作流。

Phase 2: 节点与后端服务的绑定

将后端现有的 API 封装为标准节点。例如，将 `prompt_learning/extractor.py` 绑定到一个“数据提取节点”，将其输入输出暴露为连线锚点 (Handles)。

Phase 3: 实时执行反馈

建立 WebSocket 通信。当用户点击“执行”时，画布上的节点应根据后端的执行进度呈现不同的状态（排队中、执行中、成功、失败）。这能极大提升复杂系统的调试效率与用户掌控感。